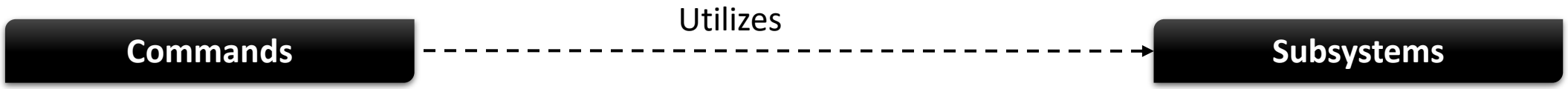


ADAMBOTS

Command Based Programming

2020 Command Based Approach

Commands and Subsystems



High-level robot actions that may utilize methods defined by the Subsystems

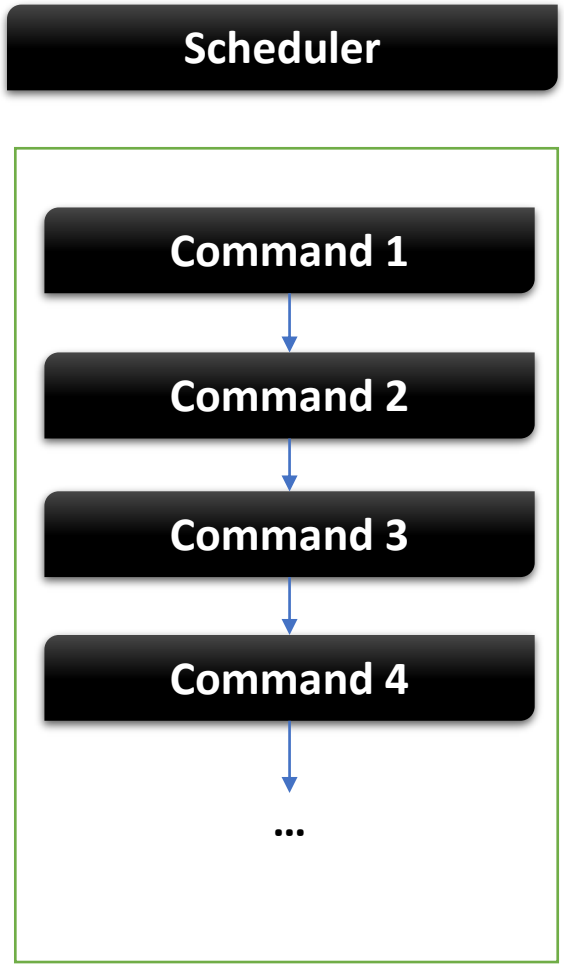
Examples:
- DriveForward
- ShootBall
- IntakeBall

Basic unit of robot organization.

Examples:
- DriveTrain
- Intake
- Elevator
- Shooter

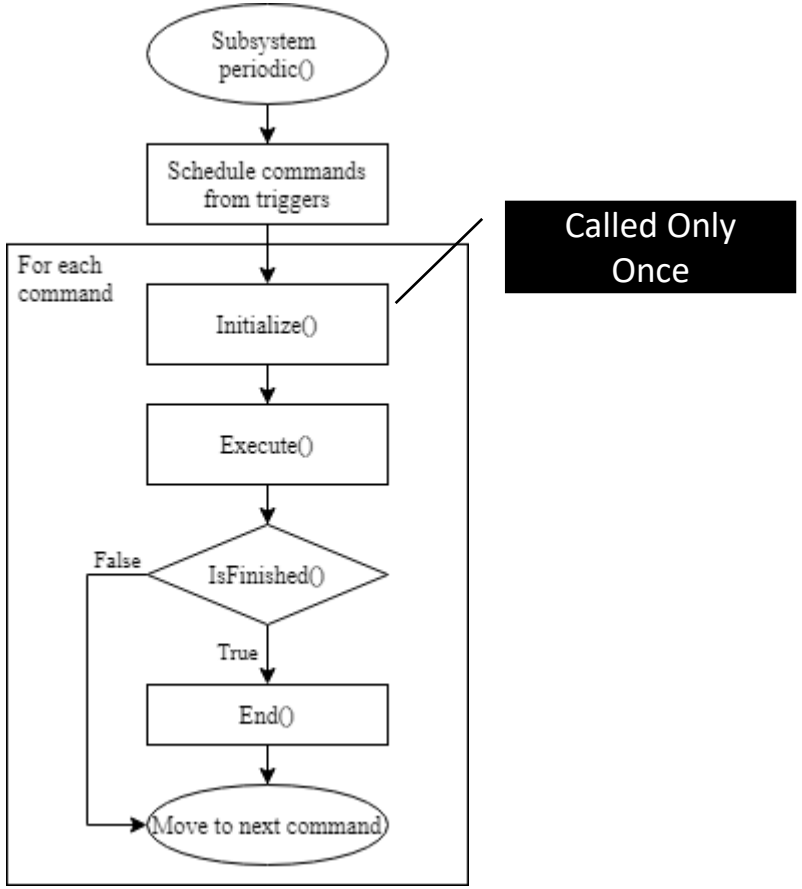
```
driveForwardCommand
aButton.whenPressed(intakeSubsystem::run);
```

Command Scheduler




```
Robot.java - robotPeriodic()

CommandScheduler.getInstance().run();
// Runs every 20 ms (50 cycles / sec)
```



Initial Setup

Robot.java

```
@Override
  public void robotInit() {

    m_robotContainer = new RobotContainer();
  }

@Override
  public void robotPeriodic() {

    CommandScheduler.getInstance().run();
  }
```

RobotContainer.java

```
public class RobotContainer {
  // The robot's subsystems and commands are defined here...
private final GondolaSubsystem gondolaSubsystem = new
         GondolaSubsystem(RobotMap.GondolaMotor);

private GondolaCommand gondolaCommand;

public RobotContainer() {

    setupDefaultCommands();
    configureButtonBindings();

}

private void setupDefaultCommands(){
   gondolaSubsystem.setDefaultCommand(
        new GondolaCommand(gondolaSubsystem,
        () -> deaden(Buttons.secondaryJoystick.getX(Hand.kLeft)))
        );
}
```

A command that will automatically be scheduled when no other command is currently using the subsystem

Triggers

- whenActive/whenPressed – when button changes from inactive to active
- whileHeld – command will be rescheduled once completed
- whenHeld – command will not be rescheduled once completed
- whenReleased – when button changes from active to inactive
- toggleWhenPressed – performs a toggle option
- cancelWhenPressed – cancels an already running command

```
private void configureButtonBindings() {

    // primary controls
    Buttons.primaryAButton.whenPressed(new ShiftLowGearCommand(driveTrainSubsystem));
    Buttons.primaryYButton.whenPressed(new ShiftHighGearCommand(driveTrainSubsystem));

    Buttons.secondaryBButton.whenHeld(new ReverseIndexToBlasterCommand(intakeSubsystem));
}
```

Subsystem Class

```
public class GondolaSubsystem extends SubsystemBase {
  /**
   * Creates a new GondolaSubsystem.
   */
  public WPI_VictorSPX gondolaMotor;

  public GondolaSubsystem(WPI_VictorSPX gondolaMotor) {
    super();

    this.gondolaMotor = gondolaMotor;
  }

  public void operateGondola(double speed) {
    gondolaMotor.set(ControlMode.PercentOutput, speed);
}

  @Override
  public void periodic() {
    // This method will be called once per scheduler run
  }
}
```

Constructor – called once when the Subsystem is created in RobotContainer

super() is important – this registers this subsystem with the Scheduler

Called once per Scheduler run

Command Class

```
public class GondolaCommand extends CommandBase {
  public DoubleSupplier gondolaSpeed;
  private final GondolaSubsystem gondolaSubsystem;

  public GondolaCommand(GondolaSubsystem gondolaSubsystem, DoubleSupplier speed) {
    this.gondolaSubsystem = gondolaSubsystem;
    gondolaSpeed = speed;

    addRequirements(gondolaSubsystem);
  }

  @Override
  public void initialize() {
  }

  @Override
  public void execute() {
    gondolaSubsystem.gondola(gondolaSpeed.getAsDouble());
  }

  @Override
  public void end(boolean interrupted) {

  }

  @Override
  public boolean isFinished() {
    return false;
  }
}
```

Constructor – called once when the Command is created

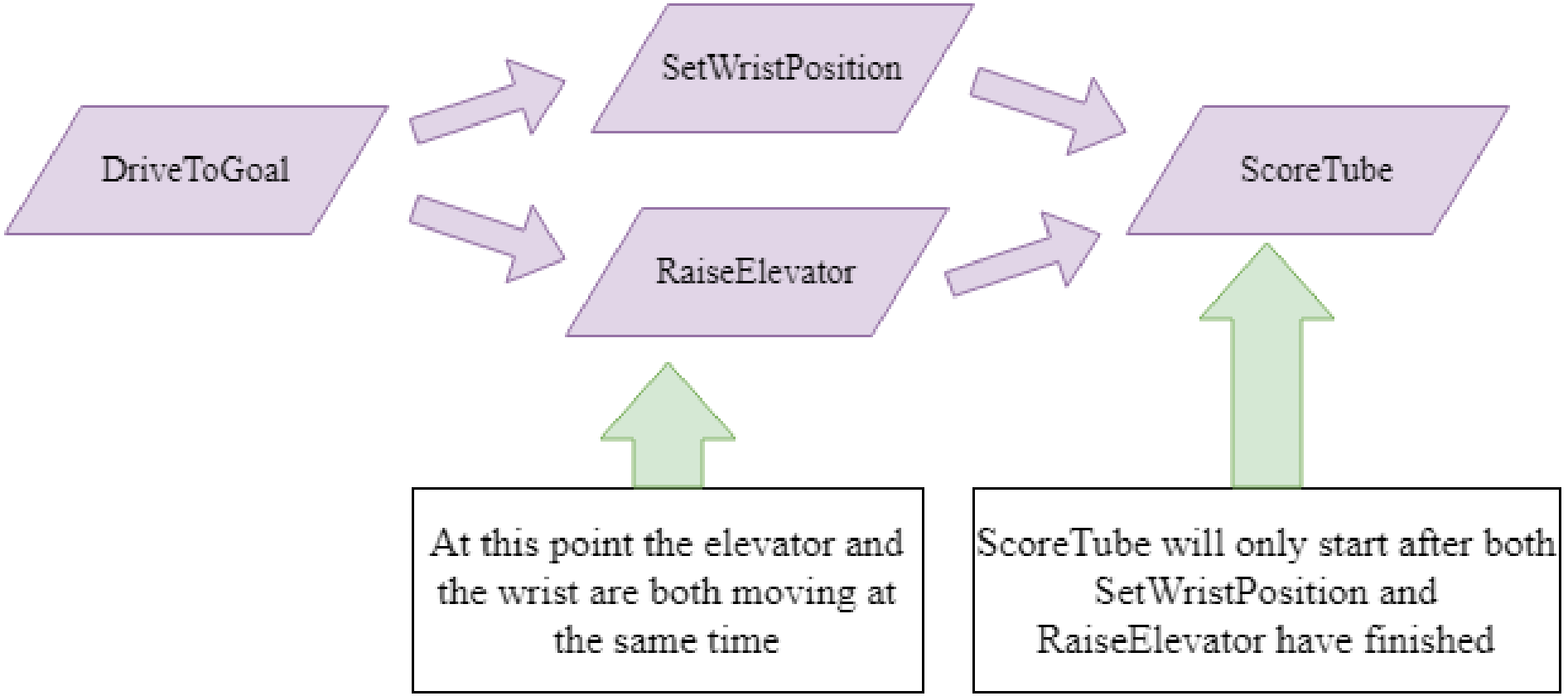
Called every time a command is scheduled to run

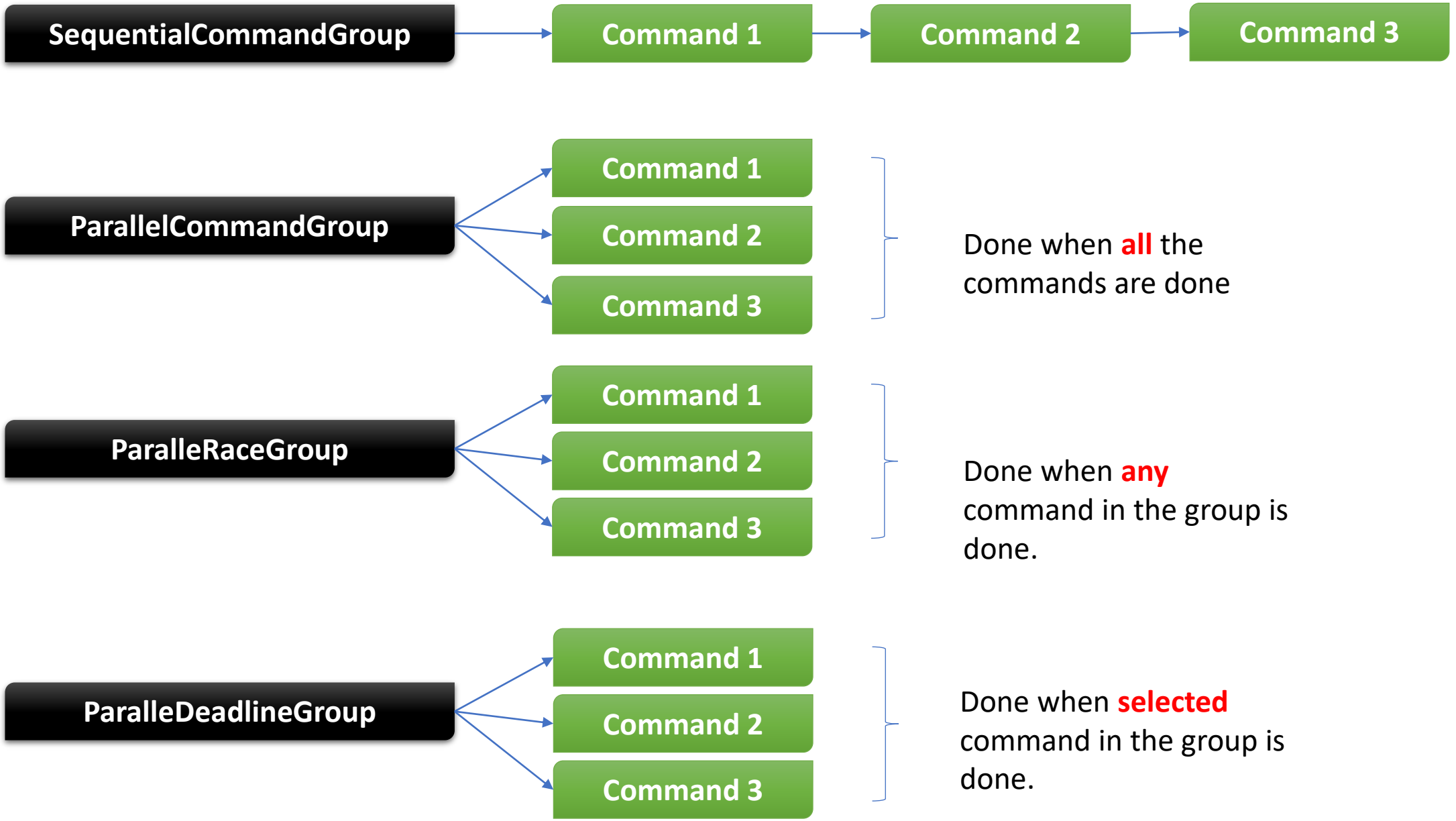
Called every time scheduler runs this command

Called every time scheduler stops this command

Command Groups

- Grouping of Commands
- Command Groups themselves are Command Objects and hence,
  - Command Groups can be passed where Command objects are accepted
  - A command group can contain other command groups




SequentialCommandGroup
Command 1
Command 2
Command 3
ParallelCommandGroup
Command 1
Command 2
Command 3
Done when all the commands are done
ParalleRaceGroup
Command 1
Command 2
Command 3
Done when any command in the group is done.
ParalleDeadlineGroup
Command 1
Command 2
Command 3
Done when selected command in the group is done.

Limitations and Dependencies

- Multiple commands can run concurrently, as long as they do not require the same resources on the robot – that is, it is not dependent on the same subsystem